

U3. AP6. Cliente Mini-FTP por sockets TCP

Objetivo de la práctica

Implementar un **cliente TCP en Java** capaz de comunicarse con un **servidor Mini-FTP** ya proporcionado, siguiendo un protocolo definido, para:

- a) Listar archivos disponibles en el servidor
- b) Descargar archivos binarios
- c) Gestionar una conexión persistente

El objetivo principal es **comprender el uso de sockets TCP**, la diferencia entre **texto y binario**, y la importancia de **seguir un protocolo de comunicación**.

Protocolo de comunicación

La comunicación cliente-servidor sigue las siguientes reglas:

Comandos enviados por el cliente (texto)

- L - Solicita el listado de archivos disponibles en el servidor.
- G nombreArchivo - Solicita la descarga de un archivo.
- Q - Cierra la conexión y termina el programa.

Respuestas del servidor

Comando L

El servidor responde con **un texto** que contiene el listado de archivos (uno por línea).

Comando G nombreArchivo

El servidor responde primero con un número long:

- 0 → el archivo no existe o es incorrecto
- 0 → tamaño del archivo en bytes

Comando Q

El servidor cierra la conexión.

Funcionalidad que debe implementar el cliente

El cliente debe:

1. Conectarse al servidor mediante un Socket
2. Mostrar un **menú de opciones** por consola
3. Enviar comandos al servidor
4. Interpretar correctamente las respuestas
5. Descargar archivos binarios sin corromperlos
6. Una vez descargado un archivo, mostrar de nuevo el menú
7. Mantener la conexión abierta hasta que el usuario salga

Pistas para implementar el cliente

Tipos de streams recomendados

Usa `DataInputStream` y `DataOutputStream`:

```
new DataInputStream(socket.getInputStream());  
new DataOutputStream(socket.getOutputStream());
```

Envío de comandos

Los comandos (L, G archivo, Q) se envían como texto. Se recomienda usar:

- `writeUTF()`
- `readUTF()`

Recepción del listado

- El listado de archivos llega como **texto**
- Basta con leerlo y mostrarlo por consola
- No hay que procesarlo ni dividirlo

Recepción de un archivo (parte clave)

Leer primero un long con el tamaño del archivo

- Si el tamaño es 0:
 - Mostrar mensaje de error
 - Volver al menú
- Si el tamaño es mayor que 0:
 - Crear un `FileOutputStream`
 - Leer los bytes en un bucle
 - Guardarlos en el archivo

Recuerda:

- `read()` no garantiza leer todos los bytes de golpe
- Debes contar cuántos bytes llevas recibidos
- Usa un buffer de tamaño razonable (por ejemplo, 4096 bytes)

- Escribe solo los bytes realmente leídos:

```
fileOut.write(buffer, 0, bytesLeidos);
```

Cierre ordenado

Cuando el usuario escriba Q:

- Envía el comando al servidor
- Cierra streams y socket
- Finaliza el programa